

BoRa: LoRa over BLE

Yeming Li, Hailong Lin, Renjie Li, Jiamei Lv, Yi Gao, and Wei Dong

{liyemnets,linhl,lirj,lvjm,gaoyi,dongw}@zju.edu.cn

College of Computer Science, Zhejiang University

Abstract

Bluetooth Low Energy (BLE) and LoRa are two dominant wireless protocols for the Internet of Things (IoT), each built with specific design goals, rendering them non-interoperable. Cross-Technology Communication (CTC) enables heterogeneous communication between BLE and LoRa. Despite these efforts, existing works suffer from low throughput and short communication range. In this paper, we present a novel system named *BoRa*, which enables bi-directional communication between the COTS BLE and COTS LoRa chips. The key idea of *BoRa* is to emulate the linear frequency changes of the LoRa chirp by continuously tuning its Modulation Offset (MO), which is originally designed to compensate for the Carrier Frequency Offset (CFO) in BLE chips. *BoRa* can be readily run on COTS chips without any hardware modifications.

We implement and evaluate *BoRa* with COTS BLE and LoRa chips. The results show that *BoRa* enables bi-directional cross-technology communication between COTS BLE and COTS LoRa chips, and long-range communication between two COTS BLE chips. It achieves up to 1500m communication range while ensuring 10 Kb/s throughput in outdoor Line-Of-Sight (LOS) scenarios. Besides, *BoRa* has up to 18.54 Kb/s higher throughput and up to 98.45% lower energy consumption compared with state-of-the-art methods.

CCS Concepts

• **Networks** → **Wide area networks; Network protocol design; Wireless access networks;** • **Computer systems organization** → **Sensor networks.**

Keywords

Internet of Things, BLE, LoRa, Cross-Technology Communication, Long-range Communication

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ACM MOBICOM '25, Hong Kong, China

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1129-9/2025/11

<https://doi.org/10.1145/3680207.3723470>

1 Introduction

The recent years have witnessed a variety of heterogeneous wireless technologies, with vastly different bandwidths, communication distances, and power consumptions. For example, Bluetooth Low Energy (BLE) [16] is a dominant low-power short-range wireless protocol and LoRa is a popular low-power long-range wireless protocol. BLE uses simple Gaussian Frequency Shift Keying (GFSK) based modulation and is widely used to build Personal Area Network (PAN) applications. According to the market report [15], the annual shipment of BLE devices exceeds five billion and almost all smart devices (e.g., phones, tablets, and PCs) support BLE. LoRa [32] employs Chirp Spread Spectrum (CSS) modulation to achieve long-distance communications [25, 29, 34, 42]. In recent years, 2.4 GHz LoRa is proposed. Compared to the previous Sub-1 GHz LoRa, the 2.4 GHz LoRa offers higher global availability as the 2.4 GHz ISM band is unlicensed and universally accessible. Therefore, the 2.4 GHz LoRa is increasingly being used in Low Power Wide Area Network (LPWAN) applications [21].

If the ubiquitous BLE hardware can be re-purposed as “LoRa” hardware, it will significantly increase its communication distance and also provide useful bi-directional communication capabilities between COTS BLE and COTS 2.4 GHz LoRa. Without requiring additional hardware, which is a significant consideration given the limited space in mobile and IoT devices, this LoRa transceiver can empower numerous potential applications. For example, if COTS BLE can support CSS modulation for long-range communication, existing BLE-based IoT devices could cover the entire home area with a single-hop network, thereby facilitating smart home applications. This approach could significantly reduce costs compared to the existing multi-hop Bluetooth Mesh solution [17]. Moreover, this cross-model transceiver can achieve interoperability between COTS BLE and LoRa. For example, smart city applications, such as smart parking lots, utilize LoRa for data transmission due to its long communication range. To provide users with data access, an application-layer gateway is required. If the COTS BLE can communicate with COTS LoRa, users can directly access the data from those smart city IoT devices with their smartphones. Another example is emergency long-range communication. Imagine some people hiking in the wild and one of them is lost. They can use the COTS BLE on their smart devices (e.g., smart

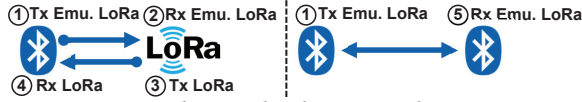


Figure 1: *BoRa* achieves bi-directional communication between COTS BLE and COTS LoRa chips, and long-range communication between two COTS BLE chips.

watch) to conduct emergency long-range communications to find each other.

State-of-the-art solutions have several limitations and have not yet fully realized this vision. In particular, these solutions either are limited to one-way communication or require Software Defined Radio (SDR) platforms with hardware modifications. Symphony [27] achieves long-range communication from BLE to an SDR gateway by directly extending the BLE symbol. It only supports one-way communication. BLE2LoRa [28] also achieves BLE-to-LoRa communication on a special BLE chip supporting 4-FSK by alternatively transmitting the four frequency shifts to coarsely emulate the linear frequency changes of CSS chirps. BLE2LoRa requires a special BLE chip and SDR gateway, and it only supports one-way communication.

In this paper, we present a novel system *BoRa*, which allows COTS BLE chips to emulate and demodulate LoRa signals. Therefore, *BoRa* facilitates bi-directional communication between COTS BLE and COTS LoRa chips, and achieves long-distance communication between two COTS BLE chips. Remarkably, *BoRa* can be readily run on existing COTS BLE/LoRa chips by just updating the firmware **without hardware modifications**. The basic idea of *BoRa* is actively fine-tuning the Modulation Offset (MO) of the transmitter to emulate the linear frequency changes of LoRa chirps. The MO tuning is an important function for COTS BLE chips that are originally designed to compensate the Carrier Frequency Offset (CFO) between the transmitter and receiver by adjusting their own carrier frequency. Although the MO tuning lays the foundation for chirp emulation, fully implementing *BoRa* still has four major challenges:

(1) Coarse-grained MO tuning granularity and timer inaccuracy on COTS BLE chips. LoRa chirps continuously sweep across the bandwidth within the predetermined chirp duration. Although the MO tuning can modify the carrier frequency, the tuning granularity is still coarse-grained compared with the frequency tuning granularity of LoRa. The emulated chirp can actually carry less data than the LoRa chirp. Additionally, COTS BLE chips do not have a high-accuracy timer that can precisely control the chirp duration. Therefore, the emulated chirps exhibit frequency and timing errors, making demodulation challenging.

(2) Strict Sync word checking on COTS LoRa receivers. COTS LoRa chips check the sync word field in the packet and filter out those packets with different sync words. This

checking process is implemented in hardware on the chip, making it an intrinsic part of the architecture that cannot be bypassed. Only packets with extremely accurate sync word chirps can be detected. However, due to the hardware limitations of BLE chips, accurately emulating these strict sync words is infeasible, leading to emulated packet loss.

(3) Unaccessible demodulation results on COTS LoRa receivers. To ensure that COTS LoRa receivers can correctly demodulate the emulated chirps, *BoRa* requires accessing the demodulation results and mapping them to the correct symbols. However, COTS LoRa hardware will demodulate emulated chirps as native symbols and directly input them into the default decoding process for further processing. As a result, the application layer can only obtain the underlying demodulation results. Therefore, how to recover the demodulation results along with the payload bytes is challenging.

(4) Direct Current (DC) distortion on COTS BLE receivers. With the directional finding proposed in Bluetooth specification 5.3 [16], COTS BLE chips can access the received I/Q samples, which can be used for LoRa signal demodulation. However, the radio frontend of COTS BLE chips is designed for receiving the GFSK modulated signal, which mainly contains the Alternating Current (AC) components. The DC component in the BLE received signal has severe energy distortion. However, the DC component is critical for LoRa chirps since each chirp crosses the zero-frequency. How to receive the LoRa signal on COTS BLE chips against the DC energy distortion is challenging.

BoRa presents four key techniques to solve these challenges. First, we propose a novel *chirp emulation method with MO tuning*. Specifically, the BLE chip generates a stable frequency signal by continuously transmitting data 1 or 0 and tunes the MO to emulate a LoRa chirp that starts with an initial MO and frequency linearly changes with time. For frequency inaccuracy, a noise-resilient symbol mapping strategy is applied to map erroneous demodulated symbols to the correct ones according to the frequency differences. For the timer inaccuracy, *BoRa* addresses the timing error introduced by each timer interrupt and dynamically compensates for the error to ensure the accuracy of the duration of emulated chirps. With this technology, *BoRa* can accurately emulate chirp signal on COTS BLE chips. Second, even with the accurate chirp emulation method, the emulated chirp still has unavoidable errors due to hardware limitations, emulated packets can still be discarded by COTS LoRa chips because of LoRa's strict sync word checking.

To enable COTS LoRa chips to successfully detect the emulated LoRa packets, we propose a *symbol difference-tolerant emulated packet detection method*. It fully exploits the underlying packet detection mechanism of COTS LoRa and designs the emulated sync word chirp that minimizes the frequency difference. Besides, it improves the symbol error tolerance

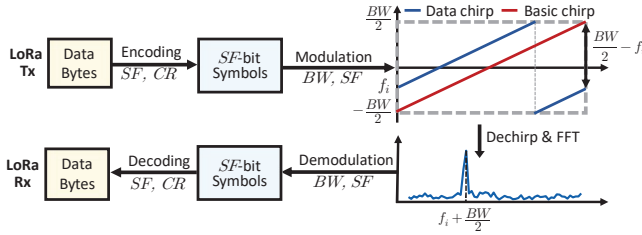


Figure 2: The LoRa communication process.

of the sync word checking on COTS LoRa chips. Therefore, emulated chirps with unavoidable frequency errors can still be successfully detected. Third, to recover the demodulation results on COTS LoRa chips for symbol mapping, we propose a *software-defined symbol recovery method*. It reverses the decoding processes in the COTS LoRa chip and recovers the physical layer demodulated symbols. Finally, to receive full LoRa signals on COTS BLE chips without DC energy distortion, we propose a *chirp reception method with DC elevation*. It deliberately introduces CFO between the transmitter and the BLE receiver. Therefore, the frequency range of LoRa chirp can avoid zero frequency and prevent any energy loss.

We implement *BoRa* with the COTS BLE (TI CC2400 [39]) and COTS LoRa (Semtech SX1280 [36]) chips, and open source our code¹. We extensively evaluate the performance of *BoRa* and build two real-world testbeds to show the practical usage scenarios of *BoRa*. The results show that (1) *BoRa* enables bidirectional communication between BLE and LoRa: achieves up to 18.54 Kb/s higher throughput and up to 98.45% lower energy consumption compared with state-of-the-art methods [27, 28] from BLE to LoRa. Moreover, *BoRa* enables LoRa to BLE communication. (2) *BoRa* can achieve up to 1500m communication range between COTS BLE and COTS LoRa chips, and up to 1100m communication range between two COTS BLE chips, while ensuring 10 Kb/s throughput in outdoor Line-Of-Sight (LOS) scenarios.

2 Preliminary and Backgrounds

2.1 LoRa Communication Process

LoRa uses a modulation scheme called Chirp Spread Spectrum (CSS). In CSS, data is modulated into chirps, whose frequency continuously sweeps across a specific bandwidth. Fig. 2 presents the process of LoRa to transmit and receive six-byte data. In the transmitter, given the bandwidth (BW), spreading factor (SF), and coding rate (CR), LoRa symbols are modulated by shifting the Initial Frequency Offset (IFO) of the time-varying frequency sequence. Generally, there are 2^{SF} available IFOs between $[-BW/2, BW/2]$. The i -th IFO (f_i) corresponds to transmitting the data $i \in [0, 2^{SF-1}]$, while the chirp with $f_i = -BW/2$ is the basic chirp.

To demodulate the LoRa chirps, the receiver first dechirps the received signal by multiplying it with a down-chirp,

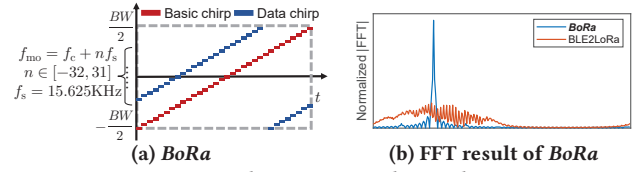


Figure 3: *BoRa* emulates LoRa chirps by MO tuning.

which is the conjugate of the basic chirp. Intuitively, the dechirp result is a single tone with a frequency equal to $f_i + BW/2$. However, in LoRa modulation, the cyclical frequency shifting can lead to two different frequencies. For example, the frequency of the data chirp in Fig. 2 that exceeds $BW/2$ is downshifted to $-BW/2$. Therefore, dechirp result contains two frequencies: $f_i + BW/2$ and $f_i - BW/2$. As depicted in Fig. 3(b), practically, the receiver can add up the Fast Fourier Transform (FFT) results within the intervals $[-BW, 0]$ and $[0, BW]$ to get a prominent frequency peak at $f_i + BW/2$. After that, the LoRa receiver can demodulate the SF -bit symbol corresponding to the chirp with the location of the frequency peak and finally decode all the received symbols to the payload data.

2.2 BLE Modulation

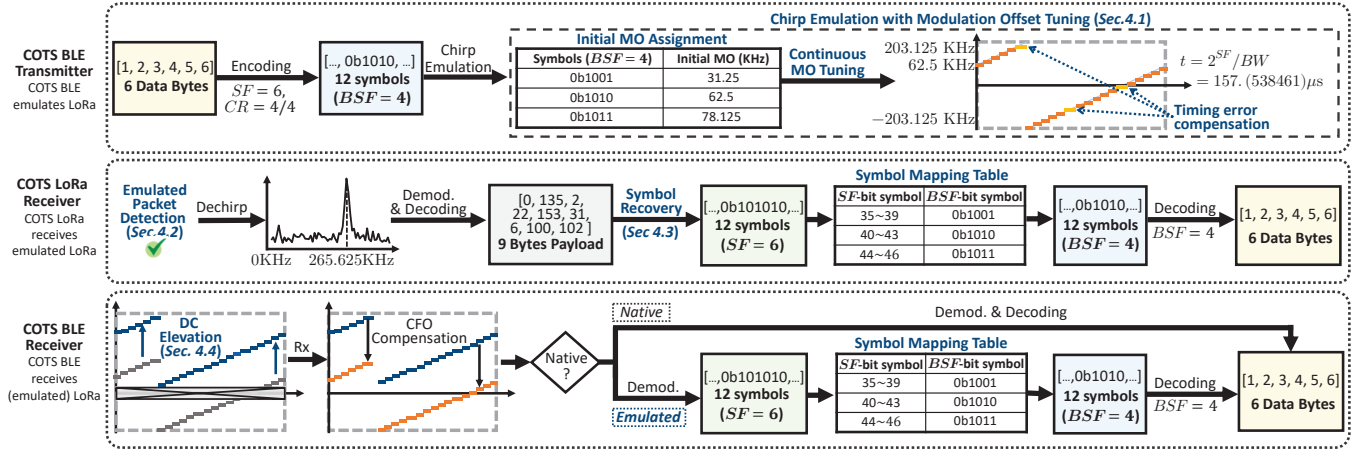
BLE utilizes Gaussian Frequency Shift Keying (GFSK) modulation [16]. The frequency deviation f_{dev} is positive while transmitting bit 1 and negative for bit 0. BLE typically sets $f_{dev} = \pm 250$ KHz. After the GFSK modulation, the baseband signal will be loaded on a carrier wave with center frequency f_c for transmission, and the receiver can remove the carrier wave to get the baseband signal for demodulation.

Because of the hardware imperfection, the carrier frequencies generated on the transmitter and receiver are different, and the frequency difference is denoted as CFO (f_{cfo}). The CFO can impact the GFSK signal demodulation. Many COTS BLE chips have CFO compensation functions, such as TI CC2400 [39], CC2500 [4], and Silicon Lab EFR32xG1 [37]. The CFO compensation is achieved by tuning the MO, denoted as f_{mo} . The center frequency of the received signal after compensation is $f_{cfo} - f_{mo}$. Similarly, CFO compensation is also performed to make the transmitted frequency match the carrier frequency of the receiver. If the f_{mo} is properly set, the compensated signal can eliminate the impact of CFO, thereby enhancing the demodulation performance of GFSK.

2.3 Basic Idea

BoRa utilizes the MO tuning mechanism supported in COTS BLE chips to achieve fine-grained chirp emulation. Specifically, it first lets the COTS BLE chip transmit a constant frequency by continuously transmitting data bit 1 or 0, and then dynamically fine-tune the MO in real time to linearly change the transmit signal frequency to emulate LoRa chirps. Take the TI CC2400 [39] for example, the chip allows the

¹<https://github.com/sada45/BoRa>

Figure 4: System overview of *BoRa*.

user to change the MO from -500 KHz to 484.375 KHz with 15.625 KHz granularity (Fig. 3(a)). Through the continuous increase of MO, *BoRa* can realize fine-grained chirp emulation with 7.8125 KHz average frequency error. In the FFT result (Fig. 3(b)), there is a prominent energy peak near the f_i and the side lobes can attenuate quickly, while BLE2LoRa [28] experiences a severe frequency leakage since it only uses four different frequencies to emulate LoRa chirps.

3 System Overview

3.1 System Architecture

BoRa enables COTS BLE chips to emulate LoRa chirps and achieve Cross-Technology Communication (CTC) between LoRa as well as long-range communication between BLE. Fig. 4 shows the system overview of *BoRa* and key workflows of *BoRa* in terms of different roles.

(1) COTS BLE Transmitter

COTS BLE chips implemented with *BoRa* would transmit emulated LoRa chirps with two steps: (1) encoding the payload according to the LoRa specification in order that COTS LoRa chips decode the *BoRa*'s signal successfully, and (2) emulating chirp with MO tuning. The key for accurate chirp emulation lies in step (2), which consists of two parts, namely initial MO assignment, and time error compensation.

Initial MO (IMO) assignment. The chirp emulation starts from choosing appropriate IMOs (f_{imo}). In order to facilitate the chirp demodulation, f_{imo} should be as close as to the IFO of LoRa chirps. However, the MO tuning granularity f_g is typically larger than the LoRa IFO spacing f_s , and the available MO values are not perfectly aligned with the LoRa IFOs. For the example in Fig 4, when emulating a chirp whose $BW=406.25$ KHz and $SF=6$, f_s equals to 6.35 KHz that is smaller than the MO tuning granularity of CC2400 (i.e., 15.625 KHz [39]). Furthermore, only a few MO values are perfectly aligned to LoRa IFOs (i.e., 0 and $\pm BW/2$).

To this end, *BoRa* chirp reduces the amount of data (BSF-bit) carried by each chirp, to achieve accurate chirp emulation and complete data representation. Specifically, the number of available MO values within the bandwidth is $n_c = \lfloor BW/f_g \rfloor$ and the highest BSF is $BSF_{max} = \lfloor \log_2(n_c) \rfloor$. The duration of emulated chirp remains the same as that of the native chirp, denoted as $t_{dur} = 2^{SF}/BW$. *BoRa* has to choose the best 2^{BSF} IMOs out of all n_c available MOs that can optimize the demodulation performance with symbol mapping. The details of the IMO assignment are presented in Sec. 4.1(1).

Time error compensation. *BoRa* continuously tunes the MO within a given duration to emulate the chirp. However, the timer in COTS BLE chips is not precise enough to control the duration time of a symbol, which introduces cumulative emulation error. For example, with $BW=406.25$ KHz and $SF=6$, the symbol duration is $157. (538461) \mu s$. COTS BLE chips typically offer only a few fixed clock frequencies, such as 48 MHz for TI CC2650. *BoRa* dynamically compensates for the timing error during chirp emulation to eliminate the accumulated errors. The details of the chirp emulation are presented at Sec. 4.1(2).

(2) COTS LoRa Receiver

The LoRa receiver works with the spreading factor SF and it demodulates each emulated chirp as a SF -bit symbol. The basic idea of correctly demodulating the emulated symbols is applying symbol mapping that translates the demodulated SF -bit LoRa symbol to the BSF -bit emulated symbol with the minimal frequency difference between the corresponding IFO and IMO. For the example in Fig. 4, if the IMO of the emulated chirp is 62.5 KHz, and the LoRa receiver is likely to demodulate the emulated chirp as the four SF -bit symbols with f_i between 50.78 KHz and 69.82 KHz. Considering the IMO assignment results, the closest IMO to these SF -bit symbols is 62.5 KHz, the LoRa receiver can map these four SF -bit symbols to the BSF -bit symbol $0b1010$. After symbol mapping, a standard LoRa decoding procedure can be applied

to get the correct payload. Due to the black-box design of COTS LoRa chips, there are two problems need to be solved:

Emulated packet detection. The physical layer of COTS LoRa chips will check the sync word field in the packet and filter out packets with incorrect sync words. To solve this, we propose *frequency difference-tolerant packet detection*. Specifically, we employ reverse engineering to understand the details of sync word checking in the COTS LoRa chip and enhance the tolerance to sync word errors. We present the details of this technology in Sec. 4.2.

Symbol recovery. Once the COTS LoRa chip received the LoRa signal, the COTS LoRa chip will conduct signal demodulation and decoding. We can only access the final decoding results and cannot access the underlying demodulation results. Therefore, we cannot map the demodulated symbols to the correct ones. To overcome this problem, we propose *software-defined symbol recovery technology* to get the demodulated symbols. The process of symbol recovery is entirely implemented in software at the application layer, requiring no hardware modifications. The details of symbol recovery is presented in Sec. 4.3.

In summary, the workflow of COTS LoRa receivers is as follows: 1) Performing frequency difference-tolerant packet detection that improves the symbol error tolerance of sync word checking to improve the detection ratio of emulated packets; 2) Demodulation and decoding chirps, which are hardware-controlled; 3) Performing software-defined symbol recovery that reverses the decoding process of LoRa to recover the demodulated SF -bit symbols; 4) Mapping the demodulated SF -bit symbols to BSF -bit symbols according to the symbol mapping table; 5) Performing the original LoRa decoding process to get the data.

(3) COTS BLE Receiver

The Bluetooth specification 5.3 proposes the direction finding [16], which allows the COTS BLE chip to access the received I/Q samples. *BoRa* access the received I/Q samples and perform FFT to demodulate the emulated chirps. However, the radio front end of the COTS BLE chip is not designed for chirp receiving. There is energy distortion in the DC component, which severely impacts the demodulation performance. Therefore, we propose *DC elevation-based CSS signal receiving method*. It introduces a fixed CFO between the transmitter and receiver, making sure that the frequency range of the chirp avoids the DC component to prevent energy distortion. The details are presented in Sec. 4.4.

Generally, the workflow of the COTS BLE receiver is: 1) Receiving CSS signal with DC elevation; 2) Symbol demodulation. For native LoRa, *BoRa* just applies the standard LoRa demodulation. For emulated LoRa signals, *BoRa* first demodulates the signal and applies the symbol mapping to get the emulated symbols. 3) Performing the standard LoRa decoding procedure to retrieve the payload.

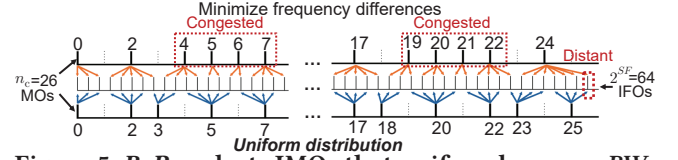


Figure 5: *BoRa* selects IMOs that uniformly across BW.

4 System Design

4.1 Chirp Emulation with MO Tuning

To transmit emulated LoRa packets, the COTS BLE chip first encodes the payload data into BSF -bit symbols with the standard LoRa coding scheme. Then, *BoRa* transmits emulated chirps corresponding to the symbols. There are two challenges for the chirp emulation that need to be solved. First, how to select the best 2^{BSF} MOs as the IMOs that optimize the performance of the COTS LoRa chip receiving the emulated chirps? Second, how to precisely fine-tune the MO to emulate chirps with limited timer accuracy?

(1) Noise-resilient IMO assignment. The emulated chirps start from the IMO to carrier data. The IMO assignment is to select the 2^{BSF} IMOs from n_c candidate MO values for all the BSF -bit symbols. For the example in Fig. 4, *BoRa* needs to select $2^{BSF} = 16$ appropriate IMOs from all $n_c = 26$ available MOs within 406.25 KHz bandwidth. The goal of the assignment is to ensure both optimal and fair performance in the COTS LoRa chip as it receives the emulated chirps with symbol mapping. Considering the LoRa receiver will demodulate the emulated BSF -bit symbol to a SF -bit symbol that has the closest IFO with the IMO. An intuitive assignment strategy is to minimize the frequency difference between the IMOs and IFOs. However, we find the minimal frequency differences to any LoRa IFOs change periodically with the MO changes, which makes the selected IMOs nonuniform on all candidate MOs. An example is shown in the upper part of Fig. 5, which illustrates the IMO assignment and the symbol mapping. Some of the chosen MOs (e.g., 4~7 and 19~22) are congested. Specifically, the congested MOs will compete with each other to map to the closely related IFOs. Therefore, some IMOs correspond to very few IFOs. (e.g., 5, 6th IMOs in Fig. 5). Such nonuniformity can lead to two shortcomings: (1) High Symbol Error Rate (SER). Some of the IMOs have few corresponding IFOs (e.g., 5th, 6th, and 21st MOs), leading to poor symbol mapping performance. (2) Low bandwidth utilization. Some IMOs are distant from their corresponding IFOs, resulting in the wastage of some IFOs. For example, the last IFO in Fig. 5 is 24.90 KHz away from the 24th MO. The LoRa receiver rarely demodulates emulated chirps as the last SF -bit symbol, so the last IFO is wasted.

BoRa selects IMOs to achieve as uniform a distribution as possible across the entire bandwidth. An example is shown in the lower part of Fig. 5, this strategy avoids selecting more than two consecutive MOs, so the number of IFOs mapped to

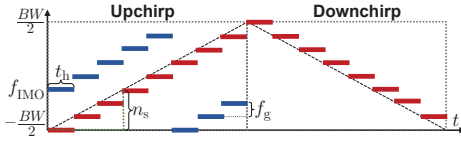
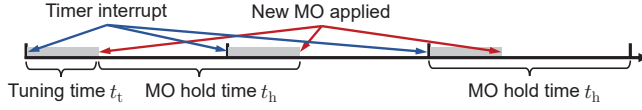
Figure 6: The MO tuning timing of *BoRa*.

Figure 7: MO tuning timeline for chirp emulation.

each IMO is the most balanced. This strategy can fully utilize the entire bandwidth to optimize symbol mapping performance. The maximum frequency tolerance of each emulated symbol is between 12.94 KHz and 18.31 KHz and no IFO has a large frequency difference with the corresponding IMO. The maximum frequency difference in the example of Fig. 5 is no more than 15.14 KHz. Some IMOs can be positioned close to the midpoint between two IFOs. This can be corrected through symbol mapping, as demodulating any of the nearby IFOs allows it to be mapped to the right IMO. For the example in Fig. 4, the emulated symbol 0b1011 is assigned with the 78.125 KHz IMO (18th MO), between the IFOs 76.17 KHz and 82.52 KHz, corresponding to the *SF*-bit symbol 0b101100 and 0b101101. The next IMO is 109.375 KHz (20th MO), so the IFO 88.87 KHz (46th) is closest to the 78.125 KHz. Once any of the *SF*-bit symbols between 44 and 46 is demodulated, they can be correctly mapped to the 0b1011.

(2) Fine-grained modulation offset tuning. After the IMOs are selected, *BoRa* emulates the chirp that with frequency starts with an IMO and sweeps across the entire bandwidth. Firstly, it needs to transmit a long enough signal with a stable frequency, which can be generally achieved by continuously transmitting bits 0 or bits 1. To achieve this, *BoRa* configures the BLE chip to the un-buffered mode and uses Direct Memory Access (DMA) to continuously inject fixed data bits into the BLE Tx frontend. With this, BLE can transmit arbitrary length signal with the stable frequency of $f_c + f_{dev}$ or $f_c - f_{dev}$ by injecting bit 1 or 0, respectively. It is worth mentioning that, though the stable frequency has a difference with the carrier frequency, it can be easily compensated at the receiver since the f_{dev} is fixed.

Then, within the chirp symbol duration $t_{dur} = 2^{SF}/BW$ times, *BoRa* sequentially traverses all MOs within the bandwidth to emulate the linear frequency change of the LoRa chirp. Fig. 7 illustrates the timeline of *BoRa* MO tuning. *BoRa* transmitter holds on each MO for t_h time, which is:

$$t_h = \frac{2^{SF}}{BW \cdot n_c}. \quad (1)$$

BoRa modifies the register (e.g., MDMCTRL in CC2400 [39]) of the BLE chip to achieve MO tuning. To achieve periodic tuning, *BoRa* initiates a timer that triggers an interrupt with

Algorithm 1 Timer interrupt handler

```

1: Global:  $R, TMR$ ; Input:  $t_{inc}, r_{inc}, n_p$ ;
2: tune_modulation_offset();
3:  $TMR = TMR + t_{inc}$ ;  $R = R + r_{inc}$ ;
4: if  $R \geq 10^{n_p}$  then
5:    $TMR = TMR + 1$ ;  $R = R - 10^{n_p}$ ;

```

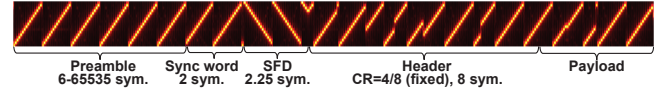


Figure 8: LoRa packet format.

t_h interval, subsequently tuning the MO upon each interrupt. It is worth mentioning that, each tuning consumes t_t time (e.g., $2.0395\mu s$ for CC2400). If the chirp duration is short, the hold time on each MO value is not enough to finish a MO tuning. To solve this, *BoRa* would skip n_s MO values and ensure the time holds on each MO for $(n_s + 1)t_h \geq t_t$.

(3) Accurate dynamic timing error compensation. As we mentioned before, *BoRa* achieves periodic MO tuning with timer interrupts. In practice, a timer interrupt is issued when the timer counter value is the same as the value of the Timer Match Register (*TMR*), which is an integer number. After each timer interrupt, *BoRa* increments the *TMR* value by t_{inc} to generate the next interrupt. However, the timer accuracy of a large number of COTS BLE chips is not enough for accurate chirp emulation. Specifically, according to Eq. 1, the t_h can be an infinite cyclical decimal for 2.4 GHz LoRa. For example, $t_h = \frac{64}{406250 \times 26}$ with $SF=6$, $BW=406.25$ KHz, and $n_c = 26$. Some COTS BLE chips can not precisely represent the t_h since they only provide some fixed clock rates, such as nRF5340 [9] with 64 and 128 MHz, and CC2650 [40] with 48 MHz. Therefore, there is a requirement to compensate for the timer error and ensure accurate emulated chirp duration.

BoRa proposes a dynamic timing compensation method, which contains two parts. First, calculate the timing error introduced by each interrupt. The *TMR* increment for each interrupt t_{inc} can be presented as $t_{inc} = \lfloor t_h/P \rfloor$, where P is the reciprocal of clock frequency. *BoRa* uses remainder increment r_{inc} to describe the timing error introduced by each interrupt, which can be presented as:

$$r_{inc} = \text{round}((t_h/P - t_{inc}) \times 10^{n_p}). \quad (2)$$

The r_{inc} accuracy is controlled by the n_p , which indicates the number of decimals that need to be reserved.

Second, accurately update the next interrupt time. The basic idea is the hold time t_h can be accurately represented by t_{inc} and r_{inc} as follows:

$$t_h = (t_{inc} + r_{inc}/10^{n_p}) \times P. \quad (3)$$

Once the timer interrupt is issued, *BoRa* will first tune the MO, increase the *TMR* with t_{inc} , and increase timer remainder R with r_{inc} . When the R is overflowed (i.e., $\geq 10^{n_p}$), *TMR* must be incremented by one to compensate for the accumulated

Table 1: Register table of Semtech SX1280 (Partial)

Address	Name	Bit	Description
0x944, 0x945	LoRa Sync Word	0:7	LoRa sync word value
0x946, 0x947	Reserved	0:7	Set 0 to change the sync word symbol error tolerance to ± 8
0x954	Header CRC	4	CRC present in LoRa incoming packet

clock errors, and R reduce by 10^{n_p} . It's worth noting that, this method supports different BLE chips by setting the P according to their specific clock frequencies. The details of the timer interrupt handler are shown in Alg. 1.

Having the above techniques, *BoRa* can now emulate chirp signals. Next, *BoRa* will emulate LoRa packets that can be demodulated by COTS LoRa receivers. Fig. 8 shows the format of the packet, which consists of a preamble, sync word, Start Frame Delimiter (SFD), header, and payload.

4.2 Emulated Packet Detection

For packet reception, the LoRa chip first performs packet detection. COTS LoRa chips would check the sync word of the packet during packet detection. Packets with non-target sync words are discarded by the receiver at the physical layer. Therefore, it is vital for *BoRa* to emulate the sync word symbols that are most similar to the target sync word symbols. To achieve this, there are two challenges: (1) Having known the target sync word, the corresponding two sync word symbols over the air are still unknown because of the black-box design of COTS LoRa chips. (2) Due to the limitations of BLE chips, the emulated chirps inevitably exhibit frequency deviations from the native sync word chirps, potentially leading to undesirable packet discarding. Different from the payload, the sync word checking is conducted by hardware and can not be corrected by symbol mapping.

To solve these two challenges, we first conduct an in-depth investigation on how the sync word works. The sync word is presented as a 1-byte number in the software. On SX1280 [36], the four Most Significant Bits (MSB) and the four Least Significant Bits (LSB) of the one-byte sync word number are stored in the four MSB of registers located at 0x944 and 0x945, respectively. Each register independently controls one sync word symbol. We directly change the values of two registers and use a USRP to receive the sync word chirps. Based on the records, there are two findings. (1) Actually, only the five bits at 7:3 of each register take effect. (2) The bits at 7:3 in the register actually represent a signed 2^2 's complement value. Denote the signed value of bits 7:3 as v , the sync word symbols is $\text{mod}(4v, 2^{SF})$. For example, the bits 7:3 is 0b10000, which presents the signed value -16, if the $SF=10$, the sync word symbol is 960.

Second, we solve the packet detection failure caused by the inevitable chirp emulation frequency error. Since the sync word checking is inherent to the chip's functionality, *BoRa*

can not apply symbol mapping to correct it. *BoRa* tries to extend the symbol error tolerance for the sync word checking on COTS LoRa chips. Although the functionality of changing symbol error tolerance of sync word on SX1280 is undocumented, we are able to pinpoint two registers (i.e., 0x946 and 0x947) related to the tolerance value among the reserved registers (Tab. 1). We set the value of the two registers to 0 and maximize the symbol error tolerance. With this, the emulated packet can be successfully detected by the LoRa receiver and for further demodulation. It's worth noting that when the COTS LoRa receiver processes a native LoRa signal, the registers can be reset to their normal values. This can reduce the symbol error tolerance and prevent potential multiplexing issues.

4.3 Demodulated Symbol Recovery

Once the emulated packet is detected, the COTS LoRa receiver will demodulate the emulated chirps to SF -bit symbols and pass the symbols through the default decoding process to get the payload bytes. *BoRa* requires the demodulated SF -bit symbols at the physical layer for symbol mapping to get the correct BSF -bit emulated symbols. However, the LoRa decoding process is conducted by hardware and can not be bypassed. *BoRa* can only access the payload bytes in the application layer. Therefore, we propose a software-defined symbol recovery technology to reverse the decoding process and recover the physical layer demodulation results.

To better illustrate our work, we first present the details of the LoRa decoding procedures in Fig. 9 [45]. Most of the procedures can be reversed. Specifically, the gary encoding can be reversed by applying gary decoding. The de-interleaving, involving matrix transpose and row-wise cyclic shifts, is reversible by inverting these operations. The de-whitening can be reversed by XORing the data stream with the whitening sequence again. CRC checking can be disabled on the COTS LoRa chip.

However, the Hamming encoding is irreversible since it is essentially a mapping from many to less. For example, the Coding Rate (CR) of the Hamming encoding is $4/8$, the encoded 8-bit data includes four data bits and four error correction bits. The Hamming decoding is mapping 256 numbers to 16 numbers. Although the LoRa chips do not officially support setting the CR to $4/4$ to disable the Hamming decoding. We find that it does, in fact, support setting the CR to $4/4$. Specifically, the value of `modParam3` of SX1280 is from 1 to 4, corresponding to $CR=4/5$, $4/6$, $4/7$, and $4/8$, respectively [36]. The CR can be set to $4/4$ by setting the value to 0. As for the eight symbols in the header, the CR is always fixed as $4/8$ even if the LoRa chip works in the implicit header mode [45]. Therefore, these eight symbols can not be recovered and *BoRa* has to discard them.

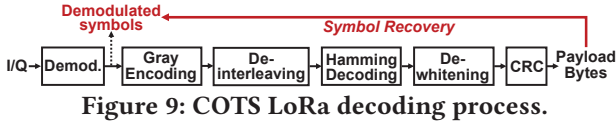
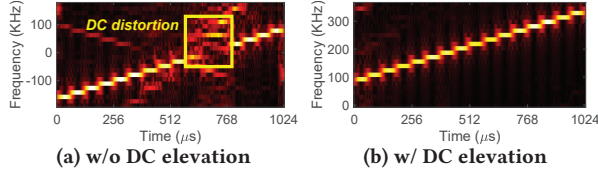


Figure 9: COTS LoRa decoding process.

Figure 10: The signal received by BLE has DC distortion. *BoRa* solves this by elevating the frequency band.

4.4 Chirp Reception with DC Elevation

BoRa enables the BLE chip to receive the LoRa or emulated LoRa signals. With directional finding, the BLE chip can access the received I/Q samples. However, we find that the DC value of the BLE received signal can be distorted. Fig. 10(a) illustrates an emulated basic chirp received by CC2400. The energy near DC is distorted and introduces considerable noise. The reason is that the BLE Rx frontend uses AC coupling to connect amplifiers [35, 38], which only allows the AC components to pass by. For the BLE, the energy loss is inconsequential since it is only concerned with the signals at frequencies $\pm f_{dev}$ for decoding, and $+f_{dev}$ for direction finding [16] (Vol. 6 Part B, 2.5.1). To solve this, we propose the DC elevation-based chirp signal-receiving method. It deliberately introduces CFO between the transmitter and receiver (i.e., the transmitter has the higher frequency). The minimum frequency of the received signal can be elevated from $-\frac{BW}{2}$ to above zero (Fig. 10(b)), which contains a clear chirp signal. The introduced CFO for DC elevation is predetermined, and both the transmitter and receiver are aware of this value. For example, if the BLE chip has 50KHz DC distortion and BW is 500KHz, the CFO value can be set as 300KHz. Therefore, the CFO for DC elevation can be compensated by software and move back to the $[-\frac{BW}{2}, \frac{BW}{2}]$ frequency range. After the I/Q signal is received on the BLE chip, *BoRa* compensates the CFO and demodulates symbols.

5 Implementation

Fig. 11 shows the implement of *BoRa*, specifically:

(1) BLE transmitter: We use Ubertooth One [14] as the COTS BLE hardware platform. It is equipped with an NXP LPC1756 [8] MCU and a TI CC2400 BLE chip [39]. The MO tuning granularity is 15.625 KHz, and the tuning range is between -500 and 484.375 KHz. The output power of CC2400 is 0 dBm and the external PA [5] on the Ubertooth One has about 20 dB gain and the output power is about 20 dBm [19], which adheres to the Bluetooth specification (-20 ~ 20 dBm) [16]. The chirp emulation is implemented

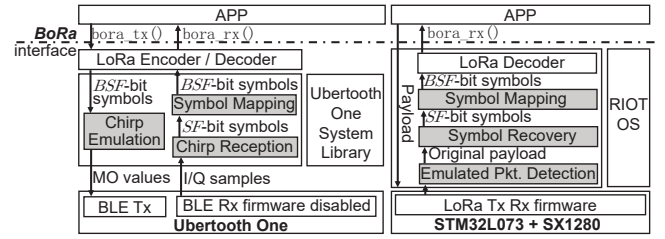
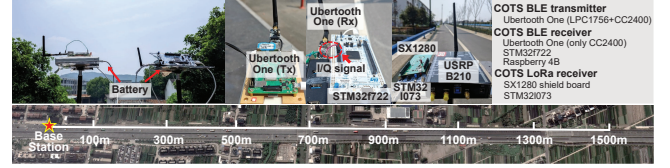
Figure 11: Implementation of *BoRa*.

Figure 12: Experimental setup (satellite map source: Google Earth).

with the Ubertooth firmware and fine-tunes the MO of the CC2400 through the Serial Peripheral Interface (SPI).

(2) BLE receiver: We use Ubertooth One as the BLE receiver. CC2400 is not the latest BLE chip, it did not natively support direction finding. But it can be set in the analog test mode, in which it can output the received analog I/Q signal to the ATEST1 (Q) and ATEST2 (I) pins. Ideally, the MCU on the Ubertooth One utilizes the ADC to collect the I/Q samples and demodulate the chirp signals. The application layer can synchronously invoke *bora_rx()* to receive the payload data. However, the ADC of the on-board LPC1756 only supports 200 Ksps sampling rate [8], much lower than modern BLE chips (e.g., nRF5340 SoC has up to 8 Msps [9]), and it is not enough for CSS demodulation. Therefore, in practice, we use the CC2400 on Ubertooth One as the Rx frontend and connect the ATEST1 and ATEST2 to an STM32f722 MCU [11], which has three independent ADCs with sampling rate up to 2.4 Msps. The STM32f722 used with RIOT OS [1] and the collected I/Q samples will be transferred to a Raspberry Pi 4B [10] for demodulation and symbol mapping.

(3) LoRa receiver: We use an SX1280 shield board (i.e., SX1280RF1ZHP [13]) attached to an STM32l073 [12] as the experiment platform. The packet detection, symbol recovery, and symbol mapping are implemented as system modules in the RIOT OS [1]. Once the emulated packet is detected, the original payload from the physical layer is input into the symbol recovery to get the SF-bit symbols and map to the BSF-bit symbols for decoding.

Setup: We take Symphony [27] and BLE2LoRa [28] for comparison. Symphony is actually the native BLE with symbol extension. To compare with our method, we set the duration of the extended BLE symbol to be the same as the emulated CSS chirps. The BLE2LoRa requires a 4-FSK chip, while the CC2400 is inherently not. To avoid the influence of hardware differences, we still implement BLE2LoRa with CC2400 but

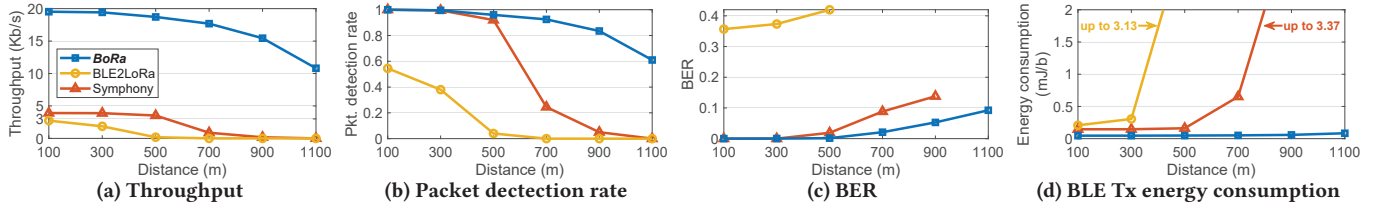


Figure 13: Performance of the COTS BLE chip to USRP.

use four MOs that are averagely distributed on the bandwidth to simulate a 4-FSK chip. These two existing works both require an SDR as the receiver. We use the USRP B210 to receive the Symphony and BLE2LoRa signals. In order to make our experimental results more instructive and reproducible, we set the carrier frequency to 2.39 GHz to avoid bursty interference from the crowded 2.4 GHz band.

6 Evaluation

In this paper, we focus on evaluating the physical layer demodulation performance. The primary performance metrics are (1) **Packet detection rate**: A LoRa packet is detected when the preamble, sync word, and SFD are all identified. For Symphony, it transmits BLE packets and each packet is detected when both the preamble and access address are detected. (2) **BER**: The ratio of the number of erroneous data bits and all data bits. (3) **Throughput**. The physical-layer data rate excludes the lost packets and erroneous bits.

6.1 Comparative Study

We first evaluate the performance with the communication distance in a road site without any coding scheme (Fig. 12). We use the Ubertooth One as the transmitter and the USRP B210 with the standard CSS demodulator [6] as the base station. We configure *BoRa* and the two related works with the same symbol duration $256 \mu\text{s}$ to fix the symbol rate. For *BoRa* we set $BW=500 \text{ KHz}$, $SF=7$, which is a commonly used configuration. For BLE2LoRa, we find it can achieve better performance with smaller bandwidth since lower bandwidth can decrease the frequency difference between its four frequency deviations and achieve relatively finer-grained emulation. Besides, even with large bandwidth, each BLE2LoRa symbol only carries two bits data. Therefore, we set $BW=250 \text{ KHz}$, $SF=6$ for BLE2LoRa. We fix the position of the base station and move the transmitter. The LOS communication range increases from 100m to 1100m.

Fig. 13(a) shows the throughput of *BoRa*. *BoRa* has 10.81 Kb/s-18.54 Kb/s higher throughput than BLE2LoRa and 10.81 Kb/s-16.81 Kb/s higher than Symphony. The reason is the packet detection rate of *BoRa* is up to 92.5% higher than the existing works (Fig. 13(b)), and BER of *BoRa* is 35.66%-41.81% lower than BLE2LoRa and up to 8.51% lower than Symphony (Fig. 13(c)). BLE2LoRa enables standard LoRa demodulators

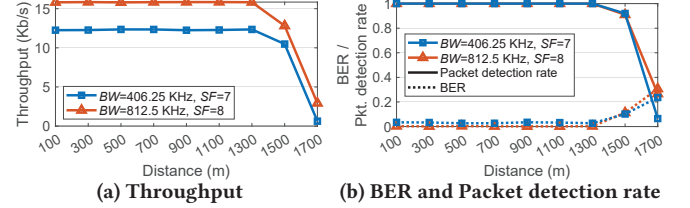


Figure 14: Performance of COTS BLE to COTS LoRa.

to receive signals transmitted by BLE. However, due to its coarse-grained emulation, the communication performance is not good even at relatively short distances (i.e., 100m). Although each symbol of Symphony only represents 1-bit data, the decoding performance seems to be better. But its BER is still higher than *BoRa*. The reason is two-fold. First, the CSS modulation fully utilizes the entire bandwidth for signal transmission, making it robust to noise [30]. While Symphony still utilizes the GFSK modulation, which only has two different frequencies (i.e., $\pm f_{dev}$). Second, each *BoRa* symbol represents BSF -bit data. When a symbol error happens, the symbol is highly likely to be demodulated as an adjacent symbol and only a few bits of the BSF -bit data represented by the symbol are incorrect. For example, the symbol 0b1010 is demodulated as 0b1011 but only has one bit error.

Fig. 13(d) shows the energy consumption to transmit one-bit data. *BoRa* requires the MCU to continuously tune the MO, resulting in relatively high MCU power consumption (i.e., 484.91 mW). The existing works only require handing over the data to the radio to complete the data transmission, and the MCU can be put into sleep mode during data transmission with only 132.80 mW power consumption. The energy consumption of *BoRa* increases from 46.39 $\mu\text{J}/\text{b}$ to 83.78 $\mu\text{J}/\text{b}$ within 1100m communication distance. It is still 77.63%-98.45% lower than BLE2LoRa and 68.09%-98.26% lower than Symphony. *BoRa* remains more energy-efficient due to its significantly higher throughput in the long-distance communication scenario, despite its higher power requirement.

6.2 Performance with COTS Devices

(1) **COTS BLE to LoRa**: We use an SX1280 as the COTS LoRa chip to receive the emulated packets. We use two configurations: (1) $BW=406.25 \text{ KHz}$, $SF=7$; (2) $BW=812.5 \text{ KHz}$, $SF=8$. Fig. 14 shows the performance of the receiving emulated signals on SX1280. Within the 1500m communication range,

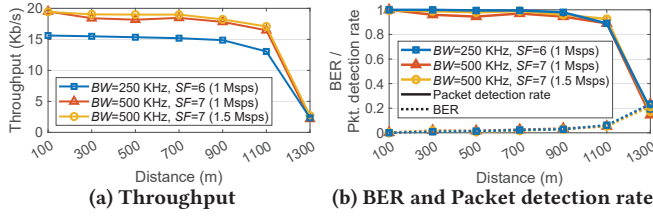


Figure 15: Performance of COTS BLE to COTS BLE via emulated LoRa signals.

the throughput of *BoRa* is higher than 10.48 Kb/s. The packet detection rate is higher than 90.95% within 1500m, and the BER is lower than 3.48% within 1300m and 10.19%–11.23% at 1500m. The COTS LoRa chip has a much better performance than the USRP receiver. Besides the slightly longer symbol duration, there are two more reasons. First, COTS RF chips have the Automatic Gaining Control (AGC) in the Rx front end. It dynamically adjusts the gain of the receiver so that the received signal has a consistent amplitude level that fully utilizes the dynamic voltage range of the ADC. However, the USRP does not have the AGC circuit and uses a fixed Rx gain (40 dB in the experiment). When the signal is weak, the amplitude does not fit with the optimal voltage range of the ADC. For example, at 1100m communication range, most of the data packet signal samples received by the USRP have amplitudes between -4.86×10^{-4} to 4.75×10^{-4} , while the amplitude range of USRP is between -1 and 1, leading to a decline in ADC sampling accuracy. Second, the COTS LoRa chip has dedicated hardware tailored for CSS demodulation, while the existing software-based LoRa demodulators are typically based on reverse engineering [45] since the details of LoRa demodulation are not fully opened.

(2) COTS BLE to BLE: We use the Ubertooth One to receive the emulated LoRa packets. Fig. 15 shows the performance of the BLE receiver. Within the 1100m communication range, *BoRa* can achieve 13.05 Kb/s–19.49 Kb/s throughput with packet detection rate higher than 89% and BER lower than 6.19%, the maximum total energy consumption is about 0.17 mJ/b. With the DC elevation, we ensure the entire frequency range of the received LoRa chirp is slightly above zero. Therefore, when the sampling rate is 1 Msps, the Nyquist frequency is 500 KHz and there is a minor loss in the high-frequency part of the chirps with $BW=500$ KHz. This can be solved by increasing the sampling rate. In the experiment, for the configuration with $BW=500$ KHz, increasing the sampling rate to 1.5 Msps from 1 Msps can result in a throughput improvement up to 0.82 Kb/s. We find that, within 1100m, the BLE receiver can achieve higher throughput than the USRP B210 receiver. This is mainly due to the signal amplification effect of the AGC in the CC2400 since the demodulation scheme is the same as the SDR receiver.

(3) COTS LoRa to BLE: We then use a SX1280 as the transmitter with $BW=406.25$ KHz, $SF=7$, and $CR=4/8$. Both a SX1280

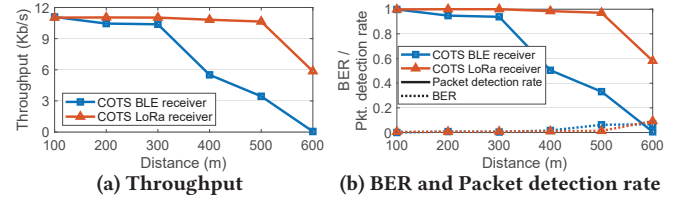


Figure 16: Performance of COTS LoRa to COTS BLE.

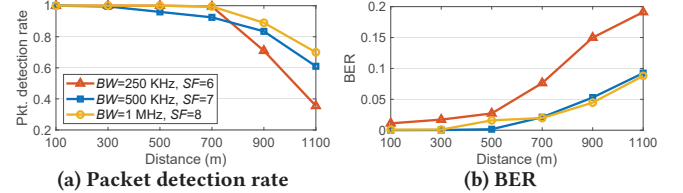
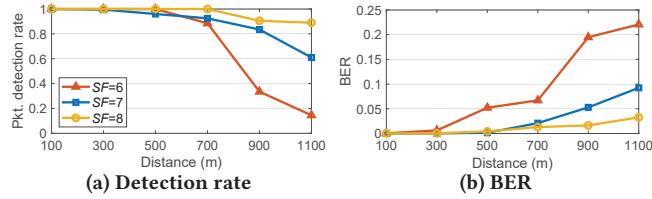


Figure 17: Impact of MO number in BW ($t_{dur}=256 \mu s$).

and an Ubertooth One are used as the receiver. We experiment on a straight road on a campus and Fig. 16 shows the results. The performance of the COTS LoRa transmitter is lower compared to the COTS BLE transmitter even if the campus road has less traffic. The reason is two-fold. First, SX1280 has a lower transmission power (i.e., 12.5 dBm) than the Ubertooth One (i.e., ≈ 20 dBm). Second, the SX1280 shield board in experiment is equipped with a PCB antenna while the Ubertooth One is equipped with an external omnidirectional antenna. Therefore, the transmitted LoRa signal is more susceptible to interference. The throughput of the COTS BLE receiver is 0.59 Kb/s–7.19 Kb/s lower than the COTS LoRa receiver when the communication range is longer than 200m. The performance degradation originates from the ADC sampling inaccuracy. Specifically, to ensure there is an integer number of sample points in each 2.4 GHz LoRa symbol duration, and the ADC sampling rate should be 1.625 MHz. However, limited by the clock precision on the STM32, accurately maintaining the sampling rate is hard. The sampling frequency can lead to packet synchronization failure and decrease the packet detection rate. This synchronization error can be accumulated and symbols farther from the preamble exhibit a higher BER. In Fig. 16, we measure the BER with short packets to alleviate the sampling rate inaccuracy issue. Using an external crystal clock is a potential solution for BLE receiving native 2.4 GHz LoRa signals. The clock can precisely control the ADC sampling to improve the performance of 2.4 GHz LoRa signal reception. However, it can lead to higher costs and more complex hardware design. We therefore consider this for future work.

6.3 Impact of Emulation Configurations

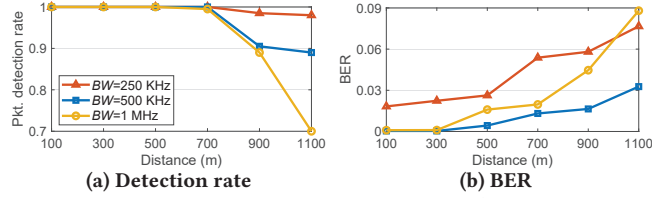
(1) Fixed symbol duration with different BW and SF: First, we evaluate the performance impact of emulation imperfection. We change the bandwidth to 250 KHz, 500 KHz, and 1 MHz, configuring the SF to fix the symbol duration to



(a) Detection rate

(b) BER

Figure 18: Impact of spreading factor (BW=500 KHz).



(a) Detection rate

(b) BER

Figure 19: Impact of bandwidth (SF=8).

256 μ s. Fig. 17 shows the experiment results. The configurations with higher bandwidth have higher packet detection rates and lower BER. The reason is that a larger bandwidth contains more MOs, improve the emulation accuracy. Specifically, there are 16, 32, and 64 available MOs within bandwidth 250 KHz, 500 KHz, and 1 MHz for CC2400 [39].

(2) Impact of SF: To evaluate the impact of the spreading factor, we fix the BW to 500 KHz and change the SF to 6, 7, and 8. Since the bandwidth is fixed, the number of available MOs is fixed ($BSF=5$ for CC2400). Fig. 18 illustrates the packet detection rate and BER of *BoRa* varying with distance. There is a trend that as the spreading factor increases, communication performance improves. A larger SF results in longer symbol duration, which facilitates more accurate detection and demodulation. Specifically, the symbol duration is 128, 256, and 512 μ s for SF=6, 7, and 8, respectively.

(3) Impact of BW: Then we fix the spreading factor to 8 and illustrate the performance impact of bandwidth. As the bandwidth decreases, the emulation becomes less perfect, but the symbol duration increases. Fig. 19 shows the experiment results. For packet detection, the smaller the bandwidth, the better the performance. This indicates that although a narrower bandwidth results in less perfect symbol emulation (less available MOs), the symbol duration is more significant for packet detection. The underlying reason is that the CSS packet detection does not carefully exploit the frequency changes. Specifically, for the preamble detection, the CSS demodulator only aims to find a sequence of continuous dechirp results with a similar frequency. Therefore, it can tolerate frequency errors between the emulated and native chirps. However, the emulation imperfection is significant for demodulation since the demodulator needs to precisely determine the frequency of each dechirped symbol. There is a trade-off for *BoRa* demodulation between the emulation imperfection and symbol duration. According to the results depicted in Fig. 19(b), the optimal bandwidth for this trade-off is 500 KHz, at which *BoRa* exhibits the lowest BER.

Table 2: Power consumption of *BoRa* (mW).

	MCU		Radio	Total
COTS BLE Tx	484.91		421.02	905.93
COTS LoRa Rx	578.30		175.60	753.90
COTS BLE Rx	STM32f722	Rasp. 4B	138.63	1303.69 (1 Msps) 1337.76 (1.5 Msps)
	655.09 (1 Msps)	509.97		
	689.16 (1.5 Msps)			

Table 3: Demodulation time of 10s signal (s).

Bandwidth (KHz)	SF=6		SF=7		SF=8	
	w/o pkt.	w/ pkt.	w/o pkt.	w/ pkt.	w/o pkt.	w/ pkt.
250 (1 Msps)	3.82	3.87	3.04	3.06	2.64	2.68
500 (1 Msps)	5.28	5.27	3.83	3.81	3.10	3.09
500 (1.5 Msps)	6.56	6.56	5.03	4.98	4.19	4.20

6.4 System Overhead

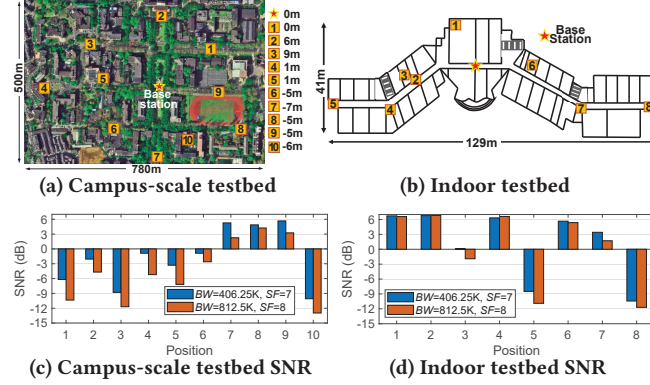
(1) Energy consumption: We use a USB power meter to measure the power consumption of *BoRa*. Specifically, we first let the Ubertooth One and LoRa receiver normally Tx/Rx data to measure the total power consumption, and we turn off the radio to measure the MCU consumption. The difference between the total power consumption and the MCU power consumption is considered as the radio power consumption. The MCU power consumption of BLE transmission includes the power of CC2400 providing the 16 MHz clock since the MCU clock on Ubertooth One is provided by the CC2400. Tab. 2 shows the result.

In general, the power consumption is acceptable. For BLE receiving the LoRa signal, we measure the power consumption of the MCU (i.e., STMf722) with two different ADC sampling rates (i.e., 1 Msps and 1.5 Msps). An increase in the ADC sampling rate results in an additional power consumption of 34.07 mW. For receiving signals with a small BW, *BoRa* can use a 1 Msps sampling rate to save energy, while a 1.5 Msps sampling rate can be used for signals with a large BW to improve performance. *BoRa* uses a Raspberry 4B with two active CPU cores to demodulate the raw signal received by the COTS BLE chip, which brings 509.97 mW extra power consumption (excluding standby power consumption). Hardware such as the Digital Signal Processor (DSP) on the MCU can be utilized to implement demodulation on embedded devices, thereby further reducing energy consumption for BLE receiving LoRa signals. We reserve this for future work.

(2) Demodulation runtime: We test the computation overhead for Raspberry Pi 4B to demodulate the I/Q signal collected from the BLE chip. Tab. 3 shows the time to demodulate 10s signals without packets or with 20 packets each. The runtime of demodulation is acceptable and resource-constrained devices can achieve real-time demodulation. The time consumption for signals with or without packets is similar. The reason is that both for packet detection and demodulation, the algorithm always splits the 10s signal into multiple t_{dur} length segments and applies FFT. Since the FFT is the most time-consuming operation and the number of

Table 4: Application runtime with chirp emulation (s).

Packet length (symbols)	No Packet	32		64	
Packet interval (s)		1	0.5	1	0.5
GCD (500,000 times)	13.4675	14.1364	14.1541	14.1439	14.1741
Fibonacci (2,000 times)	18.4331	18.4584	18.4800	18.4796	18.5146

**Figure 20: Real-world testbeds.**

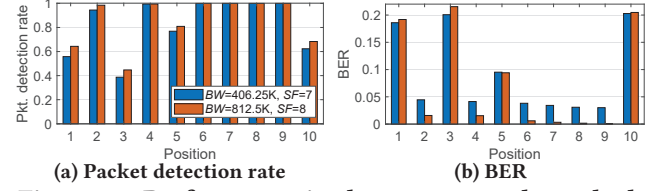
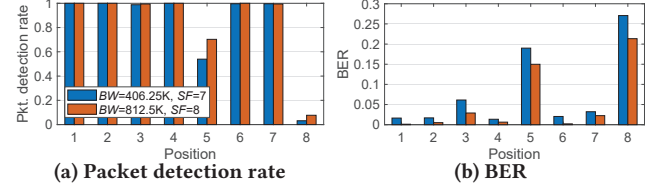
FFT is the same, the total execution time is similar. For the same BW , the time consumption reduces with SF increases. The reason is that the t_{dur} increases with SF and the segment number decreases, which improves the memory efficiency.

Impact to the application performance: To emulate chirps, *BoRa* requires the MCU to periodically generate timer interrupts and tune the MO. These interrupts can affect the applications running on the MCU. We let the Ubertooth One periodically transmit emulated chirps, while simultaneously running computational tasks. The calculation of the Greatest Common Divisor (GCD) demonstrates the multiplication/division performance and the Fibonacci number demonstrates the addition performance. Tab. 4 shows the results. With chirp emulation, the runtime of application layer programs can increase by up to 0.71s, which is acceptable.

6.5 Real-world Deployments

We evaluate the performance of *BoRa* in the real-world deployment of both outdoor and indoor scenarios. We fix the position of the COTS LoRa gateway and use BLE chips to transmit data to the gateway. The outdoor scenario experiment is conducted on a campus with undulating terrain. Fig. 20(a) illustrates the positions of the nodes and their relative elevation with respect to the gateway. The indoor scenario experiment is conducted on the first floor of a building (Fig. 20(b)). The Signal-to-Noise Ratio (SNR) of each position is shown in Fig. 20(c)(d). The SNR at $BW=406.25$ KHz is generally higher than the SNR at $BW=812.5$ KHz since the narrow band contains less noise.

(1) Outdoor scenario: Fig. 21 shows the experiment results in the campus-scale testbed. Most of the positions (4, 6, 7, 8, and 9) have good performance with packet detection rates up to 100% and BER lower than 4.14%. The position 3 has the

**Figure 21: Performance in the campus-scale testbed.****Figure 22: Performance in the indoor testbed.**

worst performance since there are some buildings between it and the base station, and there is a 9m elevation difference between them. For positions 1 and 10, the performance degradation is mainly caused by the building blockages. Positions 2 and 7 both have LOS paths to the base station with similar elevation and distance, but the performance at position 7 is slightly lower due to a large sculpture to the south of the base station (west at top of Fig. 20(a)) that block the signal.

(2) Indoor scenario: We then conduct the experiment on the first floor of a building (Fig. 20(c)) to evaluate the *BoRa* performance in the indoor scenario. We reduced the Tx power by setting the output power of the CC2400 to -10 dBm, while the gain of the amplifier remains about 20 dB. Fig. 22 shows the results in the indoor testbed. *BoRa* can achieve communication under Non-Line-of-Sight (NLOS) conditions. Except for position 5 and position 8, *BoRa* has good performance with 98.82%-100% packet detection rates and 0.11%-6.12% BER. For positions 5 and 8, which have two turns between the transmitter and the base station, the SNR can drop to as low as -11.70 dB, leading to performance degradation.

7 Discussion

Portability to other chips. In this paper, we use TI CC2400 for implementation. To fully implement *BoRa* on other COTS chips, there are three hardware constraints that need to be solved. The first is the functionality for tuning the MO. Some other chips officially support MO tuning, such as CC2500 [4], CC2511 [7], etc. The other COTS BLE chips (e.g., EFR32xG1 [37]) and other protocol chips (e.g., AT86RF231 [3], CC2420 [2]) support automatic CFO compensation but do not officially open the MO tuning function to users. We can also try to find out the registers that control the MO. Specifically, these chips typically provide a register that contains the CFO value estimated with the received preamble. We can search registers that are near the CFO estimation register. Second, some of the COTS chips do not support continuous I/Q sampling for receiving CSS signals. Though the direction finding is not

designed for data receiving, it allows users to access the I/Q sample from the antenna. However, some of the COTS BLE do not support continuous I/Q sampling. The reason is that the Constant Tone Extension (CTE) for direction finding is no longer than 160 μ s. This constraint can be possibly overcome by manually triggering specific events. For example, we can periodically trigger CTEPRESENT and END events on nRF5340 [9] to enable I/Q sampling. Third, *BoRa* requires the BLE chip to continuously transmit bits 1 or 0 for chirp emulation. Some of COTS BLE chips may not support continuously transmitting signals since they have limited transmission buffer and does not support un-buffered mode. For those chips, we need to segment one emulated packet into multiple BLE packets. Specifically, if the current BLE packet is not long enough to emulated the next chirp, the next chirp will be emulated by the next BLE packet which is Inter-Frame Space (IFS) later. We reserve these for future work.

Link-layer coding. *BoRa* reverses the decoding processes on COTS LoRa chips to receive the emulated packets. These decoding processes used in the native LoRa can be conducted by software to extend the *BoRa* communication range. Besides, we can also implement other coding schemes to further improve the performance over weak links. Such as the LT code [33], which encodes the packets into multiple segments to adapt dynamic channel conditions.

Limited communication range and throughput. For the communication performance, *BoRa* has relatively shorter communication range and lower throughput. The major reason is two-fold. First, compared with the native Sub-1 GHz LoRa, *BoRa* and 2.4 GHz LoRa have worse performance due to the fact that 2.4 GHz wireless signals are most readily absorbed by water, and their shorter wavelength makes them more susceptible to obstruction by obstacles. Second, because of the MO tuning granularity, the *BSF* is smaller than the native *SF*, emulated chirps with the same duration carry less information than native ones, leading to lower throughput.

8 Related Work

In recent years, there has been significant research focus on Cross-technology Communication (CTC) in the field of wireless communication [23, 24, 31, 46].

Some of them focus on one-way communication. Researchers use WiFi to emulate low-end wireless technologies. Notable examples include WiFi emulating IEEE 802.15.4 [22, 43], WiFi emulating BLE [18, 26], and WiFi emulating LoRa [44]. These works benefit from the high symbol rates and the utilization of high-order modulation schemes like 1024-QAM, which are incapable of operating on BLE.

Some studies explore the emulation of heterogeneous wireless protocols with low-end wireless protocols. For instance, BLE2LoRa [28], utilizes 4-FSK chips to enable BLE emulating

LoRa chirps. L2X [41] samples the Received Signal Strength Indicator (RSSI), allowing low-end wireless chips to receive LoRa signals. While these works have made contributions, their limitations, including coarse-grained emulation and the need for additional hardware, result in adequate performance and reduced practicality.

Furthermore, some works focus on achieving bi-directional communication. Unify [20] and FLEW [19] have accomplished bi-directional communication between WiFi and BLE chips. In the WiFi-to-BLE link, they emulate the WiFi signal by directly injecting data into the BLE radio front end. In the BLE-to-WiFi link, the FSK demodulator can receive the WiFi signal using Differential Binary Phase Shift Keying (DBPSK) modulation by exploiting the relative frequency offset. However, due to the distinct CSS modulation used in LoRa compared to GFSK in BLE, these approaches face challenges when it comes to enabling bi-directional communication between BLE and LoRa directly.

In general, it has been widely believed that emulating LoRa chirps on other wireless protocols is hard due to the frequency constraints imposed by the modulation schemes of the host protocols [28]. However, *BoRa* breaks away from the conventional wisdom by leveraging MO tuning to achieve fine-grained LoRa chirp emulation. Unlike previous methods, *BoRa* can receive clean LoRa chirps on COTS BLE chips with DC elevation, without requiring additional hardware.

9 Conclusion

In this paper, we present *BoRa* that leverages the modulation offset tuning functionality to continuously change the transmission frequency and emulate LoRa chirps. First, *BoRa* fully addresses the timer and frequency inaccuracy on COTS BLE chips to achieve fine-grained chirp emulation. Second, *BoRa* proposes symbol difference-tolerant emulated packet detection method and software-based symbol recovery method so COTS LoRa chip can successfully receive the emulated packets. Third, *BoRa* proposes chirp reception method with DC elevation to solve the DC distortion issue on the COTS BLE chip. We implement *BoRa* with COTS chips and conduct extensive experiments. In the outdoor LOS scenario, *BoRa* enables up to 1500m communication range between COTS BLE and LoRa, and up to 1100m communication range between two COTS BLE chips, while ensuring 10 Kb/s throughput.

Acknowledgment

This work is supported by the National Natural Science Foundation of China under grant No. 62072396, the “Pioneer” and “Leading Goose” R&D Program of Zhejiang under grant No. 2023C01033, and the National Youth Talent Support Program. Jiamei Lv and Wei Dong are the corresponding authors.

References

- [1] 2023. RIOT OS: The friendly Operating System for the Internet of Things. <https://www.riot-os.org/>.
- [2] 2024. 2.4 GHz IEEE 802.15.4 / ZigBee-ready RF Transceiver. <https://www.ti.com/lit/ds/symlink/cc2420.pdf>.
- [3] 2024. AT86RF231 Low Power 2.4 GHz Transceiver for ZigBee, IEEE 802.15.4, 6LoWPAN, RF4CE, SP100, WirelessHART, and ISM Applications. <https://ww1.microchip.com/downloads/en/DeviceDoc/doc8111.pdf>.
- [4] 2024. CC2500 Low-Cost Low-Power 2.4 GHz RF Transceiver. <https://www.ti.com.cn/cn/lit/ds/symlink/cc2500.pdf>.
- [5] 2024. CC2591 2.4-GHz RF Front End. <https://www.ti.com.cn/cn/lit/ds/symlink/cc2591.pdf>.
- [6] 2024. jkabeat/gr-lora. <https://github.com/jkabeat/gr-lora.git>.
- [7] 2024. Low-Power SoC (System-on-Chip) with MCU, Memory, 2.4 GHz RF Transceiver, and USB Controller. <https://www.ti.com.cn/cn/lit/ds/symlink/cc2511.pdf>.
- [8] 2024. LPC176x/5x User manual. https://www.nxp.com.cn/docs/en/data-sheet/LPC1759_58_56_54_52_51.pdf.
- [9] 2024. nRF5340 Product Specification. https://infocenter.nordicsemi.com/pdf/nRF5340_PS_v1.3.1.pdf.
- [10] 2024. Raspberry Pi 4. <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>.
- [11] 2024. STM32F722xx/STM32F723xx datasheet. <https://www.st.com/resource/en/datasheet/stm32f722ic.pdf>.
- [12] 2024. STM32L073x8/STM32L073xB/STM32L073xZ Datasheet. <https://www.st.com/resource/en/datasheet/stm32l073vz.pdf>.
- [13] 2024. SX1280RF1ZHP. <https://os.mbed.com/components/SX1280RF1ZHP/>.
- [14] 2024. Ubertooth. <https://github.com/greatscottgadgets/ubertooth/tree/master>.
- [15] Bluetooth SIG. 2023. 2023 Bluetooth Market Update. <https://www.bluetooth.com/2023-market-update/>.
- [16] Bluetooth SIG. 2023. Core Specification 5.4. <https://www.bluetooth.com/specifications/specs/core-specification-5-4/>.
- [17] Bluetooth SIG. 2025. Bluetooth Mesh Protocol. <https://www.bluetooth.com/specifications/specs/mesh-protocol/>.
- [18] Hsun-Wei Cho and Kang G Shin. 2021. BlueFi: Bluetooth over WiFi. In *Proc. of ACM SIGCOMM*. 475–487.
- [19] Hsun-Wei Cho and Kang G Shin. 2022. FLEW: fully emulated WiFi. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*. 29–41.
- [20] Hsun-Wei Cho and Kang G Shin. 2023. Unify: Turning BLE/FSK SoC into WiFi SoC. In *Proc. of ACM MobiCom*. 1–15.
- [21] Gwendoline Hochet Derévanckine, Alexandre Guitton, Oana Iova, Baozhu Ning, and Fabrice Valois. 2023. Opportunities and challenges of LoRa 2.4 GHz. *IEEE Communications Magazine* 61, 10 (2023), 164–170.
- [22] Xiuzhen Guo, Yuan He, Jia Zhang, and Haotian Jiang. 2019. WIDE: Physical-level CTC via digital emulation. In *Proc. of ACM IPSN*. 49–60.
- [23] Wenchao Jiang, Zhimeng Yin, Ruofeng Liu, Zhijun Li, Song Min Kim, and Tian He. 2017. Bluebee: a 10,000 x faster cross-technology communication via PHY emulation. In *Proc. of ACM Sensys*. 1–13.
- [24] Song Min Kim and Tian He. 2015. Freebee: Cross-technology communication via free side-channel. In *Proc. of ACM MobiCom*. 317–330.
- [25] Chenning Li, Hanqing Guo, Shuai Tong, Xiao Zeng, Zhichao Cao, Mi Zhang, Qiben Yan, Li Xiao, Jiliang Wang, and Yunhao Liu. 2021. Nelora: Towards ultra-low SNR LoRa communication with neural-enhanced demodulation. In *Proc. of ACM SenSys*. 56–68.
- [26] Lingang Li, Yongrui Chen, and Zhijun Li. 2021. WiBLE: Physical-layer cross-technology communication with symbol transition mapping. In *Proc. of IEEE SECON*. IEEE, 1–9.
- [27] Zhijun Li and Yongrui Chen. 2019. Achieving universal low-power wide-area networks on existing wireless devices. In *Proc. of IEEE ICNP*. IEEE, 1–11.
- [28] Zhijun Li and Yongrui Chen. 2020. BLE2LoRa: Cross-technology communication from Bluetooth to LoRa via chirp emulation. In *Proc. of IEEE SECON*. IEEE, 1–9.
- [29] Jansen C Liando, Amalinda Gamage, Agustinus W Tengourtius, and Mo Li. 2019. Known and unknown facts of LoRa: Experiences from a large-scale measurement study. *ACM Transactions on Sensor Networks (TOSN)* 15, 2 (2019), 1–35.
- [30] Boya Liu, Chaojie Gu, Shibao He, and Jiming Chen. 2023. LoPhy: A Resilient and Fast Covert Channel over LoRa PHY. In *Proceedings of the 22nd International Conference on Information Processing in Sensor Networks*. 1–13.
- [31] Ruofeng Liu, Zhimeng Yin, Wenchao Jiang, and Tian He. 2020. Xfi: Cross-technology IoT data collection via commodity WiFi. In *Proc. of IEEE ICNP*. IEEE, 1–11.
- [32] LoRa Alliance. 2023. What is LoRaWAN. https://lora-alliance.org/resource_hub/what-is-lorawan/.
- [33] Michael Luby. 2002. LT codes. In *Proc. of IEEE FOCS*. IEEE Computer Society, 271–271.
- [34] Manan Mishra, Daniel Koch, Muhammad Osama Shahid, Bhuvana Krishnaswamy, Krishna Chintalapudi, and Suman Banerjee. 2023. {OpenLoRa}: Validating {LoRa} Implementations through an Extensible and Open-sourced Framework. In *Proc. of USENIX NSDI*. 1165–1183.
- [35] Alessandra Pipino, Antonio Liscidini, Karen Wan, and Andrea Baschirotto. 2015. Bluetooth Low Energy receiver system design. In *Proc. of IEEE ISCAS*. IEEE, 465–468.
- [36] Semtech. 2024. SX1280/SX1281 Long Range, Low Power, 2.4GHz Transceiver with Ranging Capability. https://semtech.my.salesforce.com/sfc/p/#E0000000JelG/a/3n00000019OZ/Kw7ZeYZuAZW3Q4A3R_IUjhYCQEJxkuLrUgl_GNNhuUo.
- [37] Silicon Labs. 2024. EFR32xG1 Wireless Gecko Reference Manual. <https://www.silabs.com/documents/public/reference-manuals/efr32xg1-rm.pdf>.
- [38] Shuang Song, Mario Konijnenburg, Roland Van Wegberg, Jiawei Xu, Hyunsoo Ha, Wim Sijbers, Stefano Stanzione, Dwaipayan Biswas, Arjan Breeschoten, Peter Vis, et al. 2019. A 769 μ W battery-powered single-chip SoC with BLE for multi-modal vital sign monitoring health patches. *IEEE transactions on biomedical circuits and systems* 13, 6 (2019), 1506–1517.
- [39] Texas Instruments. 2024. CC2400 2.4 GHz Low-Power RF Transceiver. <https://www.ti.com/lit/ds/symlink/cc2400.pdf?ts=1708517236243>.
- [40] Texas Instruments. 2024. CC2650 SimpleLin Multistandard Wireless MCU. <https://www.ti.com/lit/gpn/cc2650>.
- [41] Shuai Tong, Yangliang He, Yunhao Liu, and Jiliang Wang. 2022. De-spread over the air: long-range CTC for diverse receivers with LoRa. In *Proc. of ACM MobiCom*. 42–54.
- [42] Shuai Tong, Zilin Shen, Yunhao Liu, and Jiliang Wang. 2021. Combating link dynamics for reliable lora connection in urban settings. In *Proc. of ACM MobiCom*. 642–655.
- [43] Shuai Wang, Woojae Jeong, Jinhwan Jung, and Song Min Kim. 2020. X-MIMO: Cross-technology multi-user MIMO. In *Proc. of ACM MobiSys*. 218–231.
- [44] Dan Xia, Xiaolong Zheng, Fu Yu, Liang Liu, and Huadong Ma. 2022. WiRa: Enabling cross-technology communication from WiFi to LoRa with IEEE 802.11 ax. In *Proc. of IEEE INFOCOM*. IEEE, 430–439.
- [45] Zhenqiang Xu, Shuai Tong, Pengjin Xie, and Jiliang Wang. 2023. From Demodulation to Decoding: Toward Complete LoRa PHY Understanding and Implementation. *ACM Transactions on Sensor Networks* 18, 4 (2023), 1–27.

- [46] Jia Zhang, Xiuzhen Guo, Haotian Jiang, Xiaolong Zheng, and Yuan He. 2021. Link quality estimation of cross-technology communication:

The case with physical-level emulation. *ACM Transactions on Sensor Networks (TOSN)* 18, 1 (2021), 1–20.